INSTITUTO DE TECNOLOGIA E LIDERANÇA (INTELI)

Arthur Tsukamoto Oliveira

Feasibility of a Hybrid Post-Quantum Transition on the Solana Blockchain via a Native Syscall

SÃO PAULO

2026

Arthur Tsukamoto Oliveira

Feasibility of a Hybrid Post-Quantum Transition on the Solana Blockchain via a Native Syscall

Final Course Project submitted to the Institute of Technology and Leadership (INTELI), as a partial requirement to obtain a bachelor's degree in Computer Science.

Advisor: Profa. Ana Cristina dos Santos
Advisor: Prof. Bryan Kano Ferreira

SÃO PAULO
2026

Cataloging in Publication
Library and Documentation Service
Instituto de Tecnologia e Liderança (INTELI)

(Cataloging record with international cataloging data, according to NBR 14724. The record will be completed later by the institution's library, after approval and before the final version is deposited.)

Sobrenome, Nome
Título do trabalho: subtítulo / Nome Sobrenome do autor; Nome e Sobrenome do orientador. – São Paulo, 2025.
nº de páginas : il.

Trabalho de Conclusão de Curso (Graduação) – Curso de [Ciência da Computação] [Engenharia de Software] [Engenharia de Hardware] [Sistema de Informação] / Instituto de Tecnologia e Liderança.
Bibliografia

1. [Assunto A]. 2. [Assunto B]. 3. [Assunto C].

CDD. 23. ed.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my advisors, Profa. Ana Cristina dos Santos and Prof. Bryan Kano Ferreira, for their guidance, availability, and valuable insights throughout the development of this work. Their support was essential at every stage of the research, from the initial definition of the problem to the experimental validation of the proposed approach.

I am also grateful to the Instituto de Tecnologia e Liderança (Inteli) and its faculty for the education, the resources, and the environment of innovation that made this research possible.

“I think I can safely say that nobody understands quantum mechanics.”

Richard P. Feynman, *The Character of Physical Law* (1965)

RESUMO

OLIVEIRA, Arthur Tsukamoto. **Viabilidade de uma Transição Pós-Quântica Híbrida na Blockchain Solana via Syscall Nativo.** 2026. 32 f. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Instituto de Tecnologia e Liderança, São Paulo, 2026.

Computadores quânticos capazes de executar o algoritmo de Shor ameaçam esquemas de assinatura digital como o Ed25519, que sustenta a autenticação de transações e componentes centrais da blockchain Solana, e viabilizam estratégias adversariais do tipo Harvest Now, Decrypt Later. Este trabalho investiga, em vez de uma migração completa da rede, uma abordagem de coexistência criptográfica híbrida, na qual o Ed25519 permanece como mecanismo nativo e o Falcon-512, esquema pós-quântico baseado em reticulados, é introduzido como camada adicional de autenticação. A proposta combina geração off-chain do par de chaves Falcon-512, vinculação on-chain da chave pública a um Program Derived Address (PDA) e deslocamento da verificação criptográfica para o runtime do validador por meio de um syscall nativo (sol_falcon512_verify), implementado em um fork do Agave. O método compara, em 30.540 transações distribuídas em lotes de 5 a 10.000, três cenários, sendo eles o Ed25519 nativo, o Falcon-512 via syscall e o Ed25519 sob arquitetura equivalente. Os resultados mostram que a verificação direta do Falcon-512 na Solana Virtual Machine (SVM) é impraticável, enquanto a via syscall é tecnicamente viável, deslocando o principal trade-off para o tamanho da transação e a integração on-chain. Sob arquitetura equivalente, o Ed25519 verificado pelo mesmo syscall atinge paridade técnica (1,03×) com o Falcon-512, com diferença de apenas 238 Compute Units concentrada na primitiva de verificação, indicando que o custo da abordagem é de origem arquitetural, e não da primitiva pós-quântica. Conclui-se que a coexistência híbrida com verificação nativa é tecnicamente plausível, deslocando o desafio de adoção para os planos arquitetural e de governança.

Palavras-chave: criptografia pós-quântica; blockchain; Solana; Falcon-512; criptografia híbrida.

ABSTRACT

OLIVEIRA, Arthur Tsukamoto. **Feasibility of a Hybrid Post-Quantum Transition on the Solana Blockchain via a Native Syscall.** 2026. 32 p. Final Course Project (Bachelor in Computer Science) – Instituto de Tecnologia e Liderança, São Paulo, 2026.

Quantum computers able to run Shor's algorithm threaten digital-signature schemes such as Ed25519, which underpins transaction authentication and core components of the Solana blockchain, and enable Harvest Now, Decrypt Later adversarial strategies. Rather than a full migration of the network, this work investigates a hybrid cryptographic coexistence in which Ed25519 remains the native mechanism and Falcon-512, a lattice-based post-quantum scheme, is introduced as an additional authentication layer. The proposal combines off-chain generation of the Falcon-512 key pair, on-chain binding of the public key to a Program Derived Address (PDA), and the shifting of cryptographic verification to the validator runtime through a native syscall (sol_falcon512_verify) implemented in a fork of Agave. The method compares, across 30,540 transactions distributed in batches from 5 to 10,000, three scenarios, namely native Ed25519, Falcon-512 via syscall, and Ed25519 under an equivalent architecture. The results show that direct Falcon-512 verification inside the Solana Virtual Machine (SVM) is impractical, whereas the syscall path is technically viable, shifting the main trade-off to transaction size and on-chain integration. Under an equivalent architecture, Ed25519 verified by the same syscall reaches technical parity (1.03×) with Falcon-512, differing by only 238 Compute Units concentrated in the verification primitive, indicating that the cost is architectural rather than stemming from the post-quantum primitive. We conclude that hybrid coexistence with native verification is technically plausible, shifting the adoption challenge to the architectural and governance planes.

Keywords: post-quantum cryptography; blockchain; Solana; Falcon-512; hybrid cryptography.

LIST OF ILLUSTRATIONS

LIST OF TABLES

LIST OF ABBREVIATIONS AND ACRONYMS

CU	Compute Units
eBPF	extended Berkeley Packet Filter
Ed25519	Edwards-curve Digital Signature Algorithm over Curve25519
FIPS	Federal Information Processing Standards
FN-DSA	FFT over NTRU Lattices Digital Signature Algorithm
NIST	National Institute of Standards and Technology
NTRU	Number Theory Research Unit (lattice family)
PDA	Program Derived Address
PQC	Post-Quantum Cryptography
RPC	Remote Procedure Call
SBF	Solana Bytecode Format
SIMD	Solana Improvement Document
SVM	Solana Virtual Machine

1 INTRODUCTION

Shor's algorithm establishes that the integer-factorization and discrete-logarithm problems can be solved in polynomial time by a sufficiently capable quantum computer [1], rendering vulnerable the public-key cryptographic schemes whose security rests on those problems. This prospect enables adversarial strategies of the Harvest Now, Decrypt Later type, in which data captured today can be exploited once quantum capability becomes available [2]. In blockchains, the main vulnerability manifests in the exposure of users' public keys on the ledger, allowing an adversary equipped with a cryptographically relevant quantum computer to derive the corresponding private key [3].

Solana relies on the Ed25519 digital-signature scheme for transaction authentication and for core components of its execution architecture [4][5]. This choice is consistent with a high-throughput network, as it combines compact signatures with low computational cost. However, because Ed25519 is based on the elliptic-curve discrete-logarithm problem, its security is no longer sufficient against quantum computers able to run Shor's algorithm [1]. Migrating to post-quantum cryptography, nonetheless, entails additional challenges. Beyond preserving operational compatibility, Solana imposes a 1,232-byte limit per transaction, which penalizes schemes with large keys and signatures [6]. In parallel, the publication of the first NIST FIPS standards for post-quantum cryptography reinforces the relevance of gradual, legacy-compatible adoption strategies [7].

In this context, Falcon-512 stands out by combining a relatively compact signature, around 666 bytes, with lattice-based post-quantum security [8], having been selected by NIST for standardization as FN-DSA. Even so, the direct adoption of this algorithm runs into a fundamental restriction of the Solana Virtual Machine. On-chain programs execute in an environment derived from eBPF/SBF, in which floating-point operations have only limited, software-emulated support rather than native support [9]. Because the Falcon-512 implementation depends on this kind of arithmetic [10], direct verification inside the SVM shows poor adherence to the environment's current restrictions.

This work therefore investigates not a complete transition of the Solana network, but an approach of hybrid cryptographic coexistence. The proposal combines Ed25519, for native compatibility, with Falcon-512 as an additional

authentication layer, binding the post-quantum public key to a Program Derived Address (PDA) and shifting cryptographic verification to the validator runtime by means of a native syscall. Accordingly, the contributions of this work can be summarized along three fronts. The first experimentally examines the limitations of Falcon-512 verification inside the Solana Virtual Machine (SVM). The second describes the integration of a native syscall for this verification. The third characterizes the effects of the approach on signing time, confirmation time, transaction size, and Compute Units consumption.

The relevance of the theme lies in anticipating, in technical and reproducible terms, a transition that the maturation of quantum computing will eventually make unavoidable for blockchains that secure financial assets. Rather than proposing yet another cryptographic scheme, the study addresses the practical question of where and how post-quantum verification can be accommodated within Solana's execution model. The remainder of this document is organized as follows. Section 2 presents the theoretical background, the related work, the rationale for the algorithm choice, the methodology, the results obtained, and their analysis. Section 3 presents the conclusions and the directions identified for future work.

2 DEVELOPMENT

2.1 Theoretical Background

2.1.1 The quantum threat to public-key cryptography

The security of classical public-key cryptography rests on computational problems that are intractable for conventional computers, such as integer factorization and the discrete logarithm. Shor's algorithm dismantles this assumption by solving both problems in polynomial time on a sufficiently capable quantum computer [1]. The consequence is not merely theoretical, since an adversary may adopt a Harvest Now, Decrypt Later posture, storing ciphertexts and public-key material today to break them once the required quantum capability exists [2]. In the blockchain setting this risk is amplified because public keys are, by design, recorded on a permanent and publicly auditable ledger. Once a public key is exposed on-chain, a cryptographically relevant quantum computer could derive the matching private key and forge the signatures that authorize the movement of funds [3].

2.1.2 Solana, Ed25519 and the transaction model

Solana adopts the Ed25519 signature scheme both for authenticating transactions and for core components of its execution architecture [4][5]. The choice is coherent with a network engineered for high throughput, since Ed25519 pairs 32-byte public keys and 64-byte signatures with low verification cost. The same design, however, inherits the quantum fragility of the elliptic-curve discrete-logarithm problem on which Ed25519 is built [1]. A further constraint shapes any migration effort, because Solana caps a transaction at 1,232 bytes [6]. Since the envelope must also accommodate account addresses, the recent blockhash, instruction data, and the signatures themselves, post-quantum schemes with large keys or signatures quickly exhaust the available budget, making naive substitution of the signature primitive impractical.

2.1.3 Falcon-512 and lattice-based signatures

Falcon-512 is a digital-signature scheme whose security derives from hard problems over NTRU lattices, and it was selected by NIST for standardization under the name FN-DSA [8]. Its main appeal for constrained environments is the comparatively small signature, around 666 bytes, against a 897-byte public key. The cost of this compactness is algorithmic, since signing and verification rely on discrete Gaussian sampling over the lattice, which requires double-precision floating-point arithmetic (IEEE 754) for numerical stability [10]. This dependence is intrinsic to the algorithm rather than a limitation introduced by any particular execution environment, a distinction that becomes central when the scheme is embedded into a virtual machine that does not provide native floating-point support.

2.1.4 The SVM execution environment and its limitations

On-chain programs in Solana run inside the Solana Virtual Machine (SVM), an environment derived from the eBPF/SBF model. Execution is metered, so every instruction and every transaction is bounded by a budget of Compute Units (CU), and programs that exceed it are aborted [9]. Critically for the post-quantum case, the environment offers only limited, software-emulated support for floating-point operations, not native execution [9]. Since Falcon-512 verification depends precisely on double-precision floating-point arithmetic [10], embedding the primitive directly inside an on-chain program is poorly suited to the SVM's current restrictions. This

mismatch motivates moving the verification out of the bytecode program and into the validator runtime.

2.1.5 Program Derived Addresses (PDAs)

A Program Derived Address (PDA) is a deterministic account address derived from a program identifier and a set of seeds, controlled by the program rather than by an external private key [18]. PDAs allow a program to own and mutate on-chain state in a way that cannot be rewritten by unauthorized accounts. In the proposed design, a PDA stores the user's Falcon-512 public key, binding it to the user's Ed25519 identity. This is the architectural device that keeps the post-quantum scheme within Solana's size budget, because the 897-byte public key resides persistently in the PDA and does not have to travel inside every transaction, which instead carries only the signature, the message hash, and the instruction metadata.

2.2 Related Work

From the standpoint of the literature, the integration of post-quantum cryptography into blockchains has already been discussed under different strategies. Yang et al. [11] show that the dominant challenge is reconciling post-quantum security with computation and communication costs. On Ethereum, LACChain reports on-chain verification of Falcon-512 signatures, but with high cost when all the logic remains in the contract layer [12]. Within the same Ethereum ecosystem, EIP-7619 proposes a precompile for generic verification of Falcon-512 signatures, while EIP-8052 proposes Falcon support through a native precompile, and both move the cryptographic operation outside the virtual machine and reduce the dependence on contracts for the verification step [13][14]. Algorand, in turn, reports post-quantum transactions with Falcon-1024, but with parameters that do not fit Solana's transaction-size limit [15].

Within Solana's own ecosystem, two recent proposals address the problem directly. Anza proposes adding Falcon-512 through the SIMD-0461 precompile, moving verification outside the SVM, but without a published experimental evaluation [16]. Jump Crypto recommends the use of native syscalls as the preferred mechanism for post-quantum integration in the validator runtime, yet without presenting a concrete benchmark [17].

Compared with these approaches, the contribution of this work lies less in proposing a new cryptographic scheme and more in adapting the verification mechanism to Solana's context, as summarized in Table 1. The focus is on a hybrid scenario in which the network's native layer remains based on Ed25519, while Falcon-512 is introduced as an additional authentication mechanism compatible with the size and execution restrictions observed in the runtime, without claiming a wholesale replacement of the network's cryptographic logic.

Table 1 – Summary comparison with related work

Approach	Integration	Execution	Main limitation
Yang et al. [11]	PQC blockchain survey	n/a	Identifies challenges but does not evaluate Solana.
LACChain [12]	Falcon-512 on Besu	Contract	High cost when verification stays in the contract layer.
EIP-7619 / EIP-8052 [13][14]	Precompile	Outside the EVM	Proposals targeted at the Ethereum ecosystem.
Algorand [15]	Falcon-1024	Native mechanism	Parameters incompatible with Solana's 1,232-byte limit.
Anza [16]	SIMD-0461 precompile	Outside the SVM	Architectural proposal, no published experimental evaluation.
Jump Crypto [17]	Falcon-512 syscall	Runtime	Recommends syscalls, no published benchmark.
This work	PDA + native syscall	Validator runtime	Evaluation restricted to a single-node environment.

Source: prepared by the authors (2026).

2.3 Algorithm Selection

The SVM executes programs in a restricted environment, with a Compute Units limit per instruction and per transaction, in addition to the constraints inherited from the eBPF/SBF model [9]. On Solana, these restrictions interact directly with the post-quantum problem. If the Falcon-512 public key and the signature were transmitted together, the total would exceed the 1,232-byte limit per transaction. In the design evaluated in this work, the public key is stored in a Program Derived Address (PDA), which reduces the transmitted payload to the signature, the message hash, and the instruction metadata [6][18]. This adjustment makes Falcon-512 the only candidate considered in this scope that remains compatible with the network limit when the public key does not travel in every transaction.

Table 2 – Algorithm compatibility with the 1,232-byte transaction limit

Algorithm	PK (bytes)	Sig (bytes)	Compatible?
Ed25519	32	64	Yes

Algorithm	PK (bytes)	Sig (bytes)	Compatible?
Falcon-512	897	666	No
Falcon-512 (PK in PDA)	n/a	666	Yes
Falcon-1024	1,793	1,280	No
ML-DSA-44	1,312	2,420	No
SLH-DSA-128f	32	17,088	No

Source: prepared by the authors (2026).

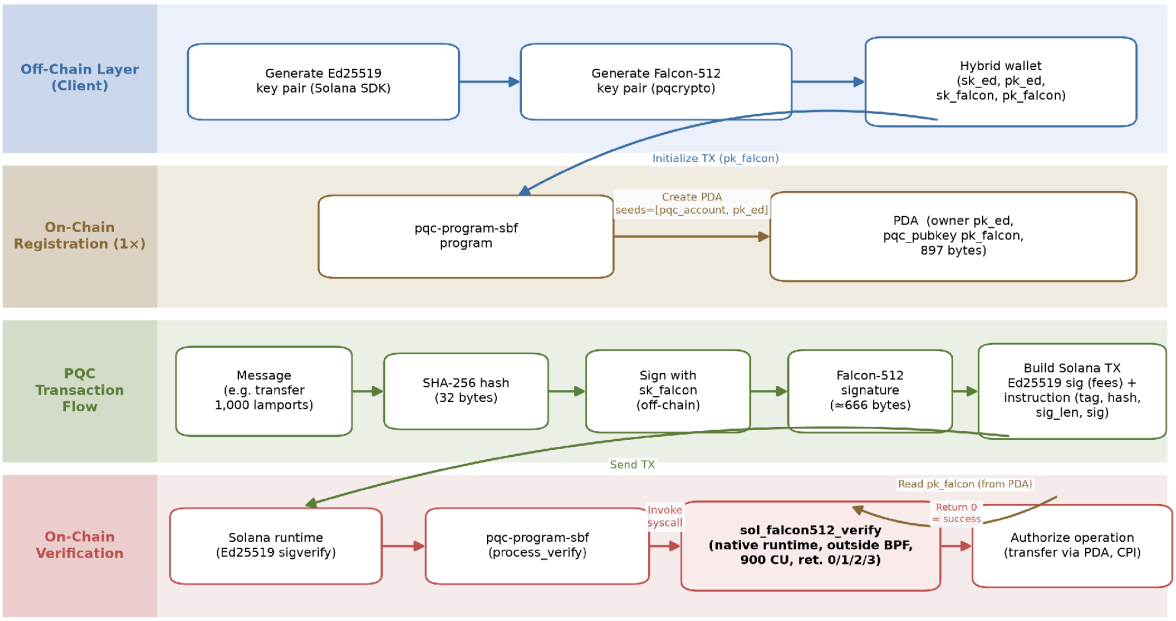
As summarized in Table 2, Falcon-512 is the only post-quantum scheme compatible with Solana when its 666-byte signature is combined with persistent storage of the public key in a PDA. Even with this optimization, the choice involves a structural trade-off, because Falcon-512 verification depends on discrete Gaussian sampling over NTRU lattices, which requires double-precision floating-point arithmetic (IEEE 754) for numerical stability [10]. This is an intrinsic characteristic of the algorithm, not a limitation introduced by the SVM's eBPF/SBF environment [9], which reinforces the inadequacy of embedding verification directly into on-chain programs without support from native runtime mechanisms.

2.4 Methodology

The study was conducted in two complementary stages. In the first, a hybrid architecture was implemented with off-chain generation of the Falcon-512 key pair, registration of the public key in a PDA derived from the user's Ed25519 identity, and submission of transactions with dual authentication. This stage aimed to show that storing the post-quantum public key in on-chain state is viable and to verify experimentally the SVM's limitation for executing Falcon-512 verification directly inside the program. In the second stage, cryptographic verification was moved into the validator runtime by means of a native syscall, allowing a direct comparison of cost and performance under controlled conditions.

The partitioning between on-chain and off-chain components stems from the 1,232-byte limit per transaction, which prevents transmitting the Falcon-512 public key together with the signature. The off-chain client generates the keys, signs locally, and builds transactions carrying only the hash, the signature, and the necessary metadata, while the on-chain program retrieves the public key from the PDA and triggers native verification. Figure 1 summarizes this division of responsibilities and the end-to-end verification flow.

Figure 1 – Hybrid post-quantum coexistence architecture and verification flow



Source: prepared by the authors (2026).

2.4.1 On-chain state and the PDA layout

The PDA account has a deterministic 955-byte layout, composed of 8 bytes of discriminator, 32 bytes for the owner's Ed25519 public key, 897 bytes for the Falcon-512 public key, 8 bytes for a creation timestamp (i64), 8 bytes for a monotonic nonce (u64), 1 byte for an initialization flag, and 1 byte for the persisted canonical bump (used in the deterministic derivation of the PDA via create_program_address). This layout cryptographically binds the user's Ed25519 identity to the stored post-quantum public key, without allowing rewrites by accounts not authorized by the program [18]. Figure 2 reproduces the constants that define this layout and the constant-cost derivation that reuses the persisted bump.

Figure 2 – Deterministic PDA layout and constant-cost derivation

```
pub const FALCON512_PUBKEY_SIZE: usize = 897;
pub const DISCRIMINATOR: &[u8; 8] = b"pqc_acct";

pub const PQC_ACCOUNT_SIZE: usize =
      8
    + 32
    + FALCON512_PUBKEY_SIZE
    + 8
    + 8
    + 1
    + 1;

let expected_pda = Pubkey::create_program_address(
    &[b"pqc_account", owner.as_ref(), &[bump]],
    program_id,
).map_err(|_| ProgramError::InvalidAccountData)?;
```

Source: implementation by the authors, fork of Agave (2026).

2.4.2 Program operations and the transfer digest

At the logical level, the hybrid program organizes the flow into three operations. The first registers the user's Falcon-512 public key and establishes the binding between their Ed25519 identity and the program-controlled PDA. The second allows a post-quantum signature to be verified without transferring funds, isolating the cryptographic step from the business logic. The third combines verification and the movement of lamports, allowing post-quantum authorization to take part in the transfer flow. In this operation, the Falcon-512 signature covers a digest that binds the recipient, the amount, the PDA, and a monotonic nonce kept in the account. The program recomputes this digest on-chain from the real transaction parameters and the current nonce, rejecting both parameter substitution and signature reuse (replay). Figure 3 shows the deterministic digest and the replay-protected transfer flow.

Figure 3 – On-chain digest and replay-protected transfer flow

```
const TRANSFER_DOMAIN: &[u8] = b"pqc_transfer_v1";

fn transfer_message_hash(
    program_id: &Pubkey, pda: &Pubkey, dest: &Pubkey,
    lamports: u64, nonce: u64,
) -> [u8; MESSAGE_HASH_SIZE] {
    solana_program::hash::hashv(&[
        TRANSFER_DOMAIN, program_id.as_ref(), pda.as_ref(),
        dest.as_ref(), &lamports.to_le_bytes(), &nonce.to_le_bytes(),
    ]).to_bytes()
}

let message_hash = transfer_message_hash(
    program_id, pqc_account.key, dest_account.key, lamports, nonce);

let result = unsafe {
    sol_falcon512_verify(
        message_hash.as_ptr(), message_hash.len() as u64,
        signature.as_ptr(), pqc_pubkey.as_ptr(), sig_len as u64)
};
if Falcon512Result::from(result) != Falcon512Result::Success {
    return Err(ProgramError::InvalidArgument);
}

let new_nonce = nonce.checked_add(1).ok_or(ProgramError::InvalidAccountData)?;
data[META_OFFSET + 8..META_OFFSET + 16].copy_from_slice(&new_nonce.to_le_bytes());

**pqc_account.try_borrow_mut_lamports()? -= lamports;
**dest_account.try_borrow_mut_lamports()? += lamports;
```

Source: implementation by the authors, fork of Agave (2026).

2.4.3 The native verification syscall

In the second stage, cryptographic verification was moved into the validator runtime by means of a native syscall, sol_falcon512_verify, implemented in a fork of Agave (the main Solana repository). The implementation, the collected data, and the scripts are publicly available in the project repository. The syscall receives pointers and lengths for the message, the signature, and the public key, maps the SVM memory, structurally validates the cryptographic components, and delegates verification to the pqcrypto-falcon library. Distinct return codes are used for success, invalid public key, malformed signature, and verification failure, allowing structural errors to be decoupled from cryptographic errors and different policies to be applied to each case in the on-chain program. Figure 4 reproduces the core of the syscall, and Figure 5 shows how both syscalls are registered behind feature gates in the runtime.

Figure 4 – Native Falcon-512 verification syscall (sol_falcon512_verify)

```
declare_builtin_function!(
    SyscallFalcon512Verify,
    fn rust(
        invoke_context: &mut InvokeContext,
        message_addr: u64, message_len: u64,
        signature_addr: u64, pubkey_addr: u64,
        signature_len: u64,
        memory_mapping: &mut MemoryMapping,
    ) -> Result<u64, Error> {
        consume_compute_meter(invoke_context, FALCON512_VERIFY_COST)?;

        let sig_len = if signature_len > 0
            && signature_len <= FALCON512_SIGNATURE_LEN as u64 {
            signature_len as usize
        } else {
            return Ok(2);
        };

        let check_aligned = invoke_context.get_check_aligned();
        let message = translate_slice::<u8>(
            memory_mapping, message_addr, message_len, check_aligned)?;
        let signature_bytes = translate_slice::<u8>(
            memory_mapping, signature_addr, sig_len as u64, check_aligned)?;
        let pubkey_bytes = translate_slice::<u8>(
            memory_mapping, pubkey_addr, FALCON512_PUBKEY_LEN as u64, check_aligned)?;

        let pubkey = match falcon512::PublicKey::from_bytes(pubkey_bytes) {
            Ok(pk) => pk,
            Err(_) => return Ok(1),
        };
        let signature = match falcon512::DetachedSignature::from_bytes(signature_bytes) {
            Ok(sig) => sig,
            Err(_) => return Ok(2),
        };
        match falcon512::verify_detached_signature(&signature, message, &pubkey) {
            Ok(())  => Ok(0),
            Err(_)  => Ok(3),
        }
    }
);
```

Source: implementation by the authors, fork of Agave (2026). Code simplified for presentation.

Figure 5 – Feature-gated registration of the native syscalls

```
register_feature_gated_function!(
    result,
    enable_falcon512_syscall,
    "sol_falcon512_verify",
    SyscallFalcon512Verify::vm,
)?;

register_feature_gated_function!(
    result,
    enable_ed25519_verify_syscall,
    "sol_ed25519_verify",
    SyscallEd25519Verify::vm,
)?;
```

Source: implementation by the authors, fork of Agave (2026).

2.4.4 Cost calibration and benchmark design

The cost of the syscall was calibrated with 10,000 measurements, following the same conversion logic used for Solana's native syscalls, with a reference of about 33 ns per Compute Unit. Table 3 summarizes the distribution of Falcon-512 verification. A value of 900 CU was adopted, equivalent to the P99 plus a 30% margin, as the conservative cost of the syscall.

Table 3 – Calibration of the sol_falcon512_verify syscall cost (1 CU ≈ 33 ns)

Metric	Time (ns)	CU
Mean	17,123	519
Median	17,042	516
Standard deviation	1,537	47
95% CI (mean)	[17,093 to 17,153]	[518 to 520]
P95	18,708	567
P99	21,917	664
P99 + 30%	28,492	863
Adopted value	n/a	900

Source: prepared by the authors (2026).

The benchmark compared three implemented approaches for transfers of 1,000 lamports on a modified solana-test-validator. The first corresponds to the native transfer authenticated by Ed25519. The second corresponds to the transfer authorized by a Falcon-512 signature verified by the native syscall. The third corresponds to Ed25519 under the same architecture as Falcon-512, verified by a dedicated native syscall. In total, 30,540 transactions were evaluated, distributed in batches of N = 5, 25, 50, 100, and 10,000, measuring signing time, confirmation time, and Compute Units. Since native Ed25519 verification is accounted for outside the program meter, the analysis distinguishes between reported cost and effective cryptographic cost to reflect this measurement disparity.

To isolate the cost of the cryptographic primitive from the cost of architectural integration, a third scenario was implemented, namely Ed25519 under the same architecture as Falcon-512 (PDA storing the public key, verification via a native syscall, and direct transfer of lamports). For this, a sol_ed25519_verify syscall symmetric to the Falcon-512 one was added to the validator runtime, with cost calibrated under the same criteria (the official SIGNATURE_COST of 720 Compute Units). In both flows, the PDA derivation is deterministic, with the canonical bump persisted in the account and verification performed by create_program_address, which guarantees constant cost. This implementation makes it possible to measure

empirically, and not merely estimate, the difference between the two flows under an identical architecture.

2.5 Results

The experiments confirmed a 100% success rate across all runs, with stable metrics in all batches (N = 5, 25, 50, 100, and 10,000), confirming that the additional cost of the Falcon-512 flow is inherent to the mechanism, and not to the experimental load. The reference values adopted for the comparison among the three scenarios are those of the largest batch, N = 10,000, synthesized in Figure 6 and detailed in Table 5.

The Falcon-512 signing time (median of 143 µs) was about six times greater than that of Ed25519, but remained on the order of microseconds and represented a small fraction of the total observed latency. The transaction size rose from 215 to 902 bytes, staying within the 1,232-byte limit. The confirmation times were statistically equivalent (median of 502 ms for the three scenarios, with the difference between Ed25519 under the same architecture and Falcon-512 being non-significant) for N = 10,000.

The decomposition of the Compute Units consumption of the Falcon-512 flow shows that, of the 7,900 total CU, only 900 CU (11.4%) correspond to the cryptographic verification performed by the syscall. The remainder is distributed among the BPF runtime overhead, reading and deriving the PDA, log calls, and other architectural operations. To ensure reproducible measurements, the PDA derivation is deterministic, with the canonical bump persisted in the account during registration and verification performed by create_program_address, which keeps the total cost constant (CV of 0.02%) and allows a direct comparison between the two flows. The logs of the two flows are identical, so the comparison reflects only the cost of the mechanism.

Table 4 details this consumption by component, measured with the sol_remaining_compute_units syscall at checkpoint points of the program, and contrasts it with the Ed25519 scenario under an equivalent architecture. The architectural components (BPF runtime, reading and deriving the PDA, logs, and transfer) are practically identical in the two flows. In absolute terms, Ed25519 under the same architecture consumed 7,662 CU against 7,900 CU for Falcon-512, a technical parity of 1.03×. The total difference of only 238 CU is concentrated mainly

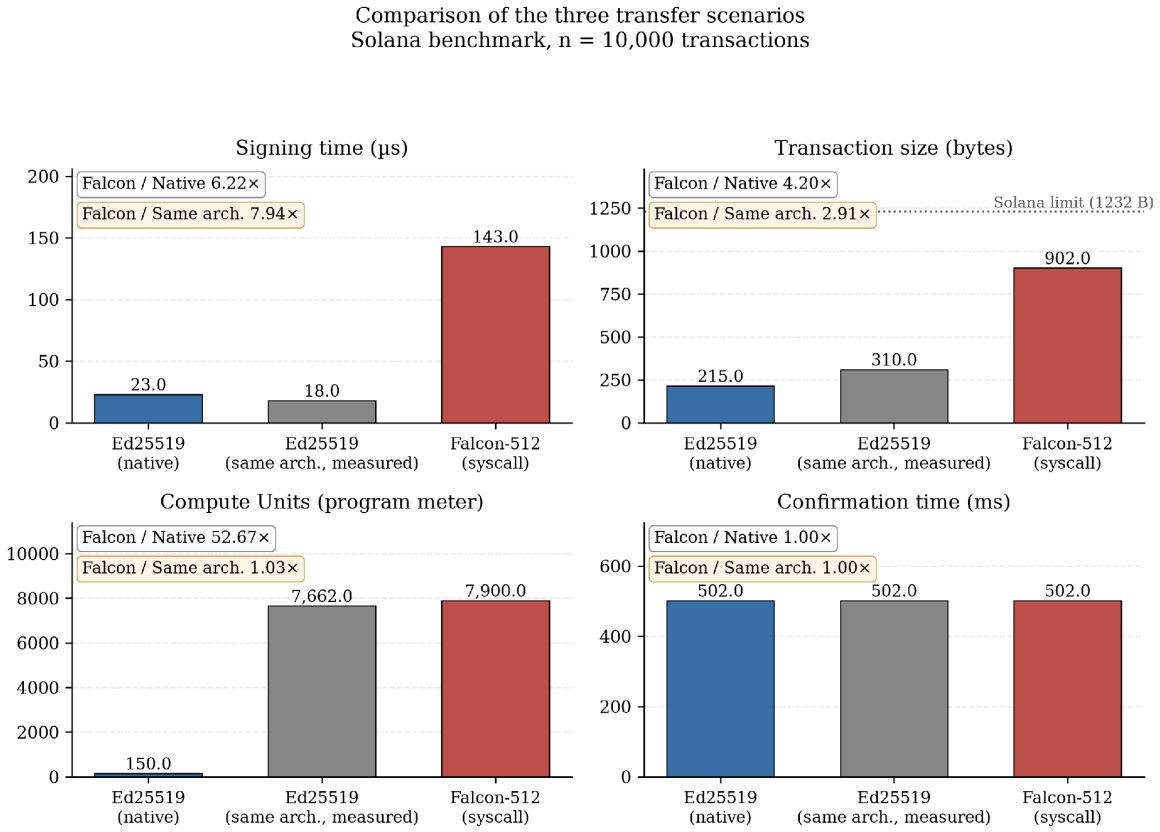
in the verification primitive (180 CU, from 720 to 900). Handling the larger Falcon-512 artifacts (666-byte signature and 897-byte public key) adds only about 58 CU. This confirms that, once the architecture is equalized, the overhead of the post-quantum flow is small and dominated by the primitive itself, rather than by the size of the post-quantum data.

Table 4 – Measured Compute Units decomposition by component (N = 10,000)

Component	Ed25519 (same arch.)	Falcon-512
BPF runtime + entrypoint	3,850	3,850
Instruction parse	21	24
PDA read + derivation (create_program_address)	1,700	1,748
Message hash + logs	723	724
Verification (primitive via syscall)	720	900
Transfer + epilogue	648	654
Total	**7,662**	**7,900**

Source: prepared by the authors (2026).

Figure 6 – Comparison of the three transfer scenarios (N = 10,000)



Source: prepared by the authors (2026). Each panel reports the associated overhead factor.

Table 5 reports the descriptive statistics per metric and scenario for the largest batch, making explicit the stability of the measurements. The Compute Units and

transaction-size metrics show negligible dispersion, while signing and confirmation times exhibit the variance expected from time measurements, yet without altering the relative ordering of the scenarios.

Table 5 – Descriptive statistics per metric and scenario (N = 10,000)

Scenario	Mean ± SD	95% CI	Md	P99
Signing time (µs)				
Ed25519	23.1 ± 3.5	[23.1 to 23.2]	23	33
Ed25519 (s.a.)	19.5 ± 18.8	[19.2 to 19.9]	18	47
Falcon-512	150.3 ± 56.6	[149.2 to 151.4]	143	298
Compute Units				
Ed25519	150 ± 0	[150 to 150]	150	150
Ed25519 (s.a.)	7,662 ± 1.9	[7,662 to 7,663]	7,663	7,663
Falcon-512	7,900 ± 1.9	[7,900 to 7,901]	7,901	7,901
Transaction size (bytes)				
Ed25519	215 ± 0	[215 to 215]	215	215
Ed25519 (s.a.)	310 ± 0	[310 to 310]	310	310
Falcon-512	902.1 ± 2.2	[902.0 to 902.1]	902	907
Confirmation time (ms)				
Ed25519	524.6 ± 104.0	[522.6 to 526.6]	502	1,005
Ed25519 (s.a.)	528.5 ± 111.9	[526.3 to 530.7]	502	1,005
Falcon-512	528.8 ± 111.9	[526.6 to 531.0]	502	1,005

Source: prepared by the authors (2026). SD = standard deviation, CI = 95% confidence interval of the mean, Md = median, s.a. = same architecture.

2.6 Analysis and Discussion of Results

Once moved into the syscall, cryptographic verification begins to exhibit behavior compatible with the evaluated environment. The isolated cost of Falcon-512 (900 CU) represents an increase of only 180 CU (+25%) over the SIGNATURE_COST of Ed25519 (720 CU), which is the direct price of quantum resistance. The total overhead of the flow, however, is much larger (7,900 CU), because most of the cost lies not in the primitive but in executing verification inside a BPF program, which involves the runtime, reading and deriving the PDA, logs, and on-chain authorization logic. This cost is practically identical to that of the equivalent Ed25519 flow (7,662 CU). The main bottleneck of the approach, therefore, is not Falcon-512 verification itself, but the integration of the verification layer into Solana's execution model.

The comparison under an equivalent architecture reinforces this interpretation. When Ed25519 is executed under the same flow (PDA, dedicated syscall, direct transfer), the measured consumption is 7,662 CU, indicating technical parity (1.03×) with the Falcon-512 flow (7,900 CU). The perceived gap between native Ed25519

(150 CU) and Falcon-512 (7,900 CU) therefore originates mostly in the transition from the native sigverify model to verification inside a BPF program (an effect of about 51×), and not in the algorithmic choice between Ed25519 and Falcon-512, whose difference under an identical architecture is only 238 CU.

The equivalence among confirmation times must be interpreted with caution, since the solana-test-validator operates without distributed consensus or multi-validator propagation. The most robust metrics are the signing time, the transaction size, and the Compute Units, which depend on the algorithm, the payload, and the runtime, rather than on the number of nodes.

2.7 Limitations and Threats to Validity

The results presented must be interpreted with reference to the single-node environment used. The solana-test-validator operates as a single node, without distributed consensus and without real propagation via Turbine or Gulf Stream. For this reason, metrics such as confirmation time offer only a partial approximation of the behavior of a production Solana network. In particular, the 4.2× increase in transaction size (from 215 to 902 bytes) may affect throughput in a multi-validator environment, but this effect could not be measured on the solana-test-validator. There is also a determinism concern relevant to consensus, since Falcon-512 verification depends on double-precision floating-point arithmetic and requires a bit-for-bit identical result across validators. This is a condition that the single-node environment does not exercise and that, on mainnet, would demand a bit-exact specification of the verification.

A scope delimitation regarding security must also be made explicit. The post-quantum guarantee applies to the assets custodied by the program, which are moved from the PDA upon Falcon-512 verification, and not to the native balance of arbitrary Ed25519 accounts. Since the payer and the transaction envelope remain signed by Ed25519, funds outside the program's control continue to be subject to the classical security model, so the coexistence explicitly protects what is placed behind the post-quantum verification.

A second limitation concerns governance and adoption on mainnet. Including the syscall in the official runtime requires approval via a SIMD (Solana Improvement Document), a vote by the majority of validators, and the update of bindings across the whole ecosystem (SDKs, wallets, RPCs, and frameworks such as Anchor). This

coordination lies outside the technical scope of the study and represents a concrete barrier between the demonstrated feasibility and effective adoption.

Finally, in the comparison under an equivalent architecture, the absolute Compute Units values depend on the runtime version and the toolchain, and the results should not be generalized to other post-quantum algorithms, to other Falcon parameterizations (such as Falcon-1024), or to other native integration strategies.

3 CONCLUSION

This work evaluated a cryptographic coexistence approach for the Solana network, in which Ed25519 remains the main mechanism and Falcon-512 is implemented as an additional authentication layer. The main contribution was to show that Falcon-512 verification directly inside the SVM faces relevant limitations, whereas its execution via a native syscall in the validator runtime proves technically feasible in the experimental environment. In addition, the results revealed that the additional cost of the approach is concentrated in the architectural overhead of integration, and not in the cryptographic primitive itself.

The comparison under an equivalent architecture reinforces this conclusion, since the measured consumption of Ed25519 under the same flow (7,662 CU) maintains technical parity (1.03×) with Falcon-512 (7,900 CU), with a difference of only 238 CU concentrated in the verification primitive. This result contradicts the perception that post-quantum schemes would be prohibitively expensive and locates the adoption challenge at the architectural and governance level, rather than at the algorithmic one.

Conducted in a single-node environment, the experiments recommend caution regarding propagation and consensus on a real network. The contribution is not to propose a complete migration of Solana, but to offer experimental evidence that hybrid coexistence with native verification is technically plausible.

Several future directions stand out. The first is the evaluation of the mechanism in a multi-validator environment, able to characterize the impact of the payload increase on throughput and propagation. The second is the direct experimental comparison with the SIMD-0461 precompile [16] under equivalent metrics. The third is the study of the governance process required to include the syscall in Solana's official runtime.

REFERENCES

[1] SHOR, P. W. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. **SIAM Journal on Computing**, v. 26, n. 5, p. 1484–1509, 1997.

[2] MOSCA, M. Cybersecurity in an Era with Quantum Computers: Will We Be Ready? **IEEE Security & Privacy**, v. 16, n. 5, p. 38–41, 2018.

[3] AGGARWAL, D.; BRENNEN, G. K.; LEE, T.; SANTHA, M.; TOMAMICHEL, M. Quantum attacks on Bitcoin, and how to protect against them. **Ledger**, v. 3, 2018.

[4] BERNSTEIN, D. J.; DUIF, N.; LANGE, T.; SCHWABE, P.; YANG, B. Y. High-speed high-security signatures. **Journal of Cryptographic Engineering**, v. 2, n. 2, p. 77–89, 2012.

[5] YAKOVENKO, A. **Solana: A new architecture for a high performance blockchain v0.8.13**. 2018. Available at: https://solana.com/solana-whitepaper.pdf. Accessed on: 15 May 2026.

[6] SOLANA FOUNDATION. **Transactions: Instruction Format**. Solana Documentation. Available at: https://solana.com/docs/core/transactions#instruction-format. Accessed on: 15 May 2026.

[7] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Post-Quantum Cryptography: FIPS Approved**. 2024. Available at: https://csrc.nist.gov/news/2024/postquantum-cryptography-fips-approved. Accessed on: 15 May 2026.

[8] FALCON PROJECT. **Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU**. Available at: https://falcon-sign.info/. Accessed on: 15 May 2026.

[9] SOLANA DOCUMENTATION. **Limitations**. Available at: https://solana.com/docs/programs/limitations. Accessed on: 15 May 2026.

[10] PORNIN, T. **New Efficient, Constant-Time Implementations of Falcon**. Technical report, 2019. Available at: https://falcon-sign.info/falcon-impl-20190918.pdf. Accessed on: 15 May 2026.

[11] YANG, Z.; ALFAURI, H.; FARKIANI, B.; JAIN, R.; DI PIETRO, R.; ERBAD, A. A Survey and Comparison of Post-quantum and Quantum Blockchains. **arXiv preprint** arXiv:2409.01358, 2024.

[12] LACCHAIN. **On-Chain Verification of Post-Quantum Signatures**. LACNet Documentation. Available at: https://lacnet.com/on-chain-verification-of-post-quantum-signatures. Accessed on: 15 May 2026.

[13] EUM, M.; LEAL, J.; VELASCO, A. **EIP-7619: Precompile Falcon512 Generic Verifier**. Ethereum Improvement Proposals, 2024. Available at: https://eips.ethereum.org/EIPS/eip-7619. Accessed on: 15 May 2026.

[14] EUM, M.; LEAL, J.; VELASCO, A. **EIP-8052: Precompile for Falcon Support**. Ethereum Improvement Proposals, 2025. Available at: https://eips.ethereum.org/EIPS/eip-8052. Accessed on: 15 May 2026.

[15] ALGORAND FOUNDATION. **Technical Brief: Quantum-Resistant Transactions on Algorand with Falcon Signatures**. 2025.

[16] RESNICK, M.; KIM, S. **Securing Solana Against a Powerful Quantum Adversary**. Anza, 2026. Available at: https://www.anza.xyz/blog/securing-solana-against-a-powerful-quantum-adversary. Accessed on: 15 May 2026.

[17] RUBIN, D.; CESENA, E.; LATIF, M. **Quantum Migration Paths for Solana**. Jump Crypto, 2026. Available at: https://jumpcrypto.com/resources/quantum-migration-paths-for-solana. Accessed on: 15 May 2026.

[18] SOLANA DOCUMENTATION. **Program Derived Addresses (PDAs)**. Available at: https://solana.com/docs/core/pda. Accessed on: 15 May 2026.

APPENDIX A – SOURCE CODE AND REPRODUCIBILITY

All artifacts required to reproduce the experiments are publicly available in the project repository, a fork of Agave (the reference Solana validator implementation). These artifacts include the native syscalls sol_falcon512_verify and sol_ed25519_verify added to the runtime, the hybrid on-chain program (registration, isolated verification, and post-quantum-authorized transfer), the benchmark and calibration clients, and the raw measurement data (CSV) for the batches of N = 5, 25, 50, 100, and 10,000 transactions. The code excerpts reproduced in Figures 2 to 5 are taken verbatim from this implementation, with logging statements omitted where indicated to improve legibility.

The repository is available at https://anonymous.4open.science/r/agave_falcon512-3C25